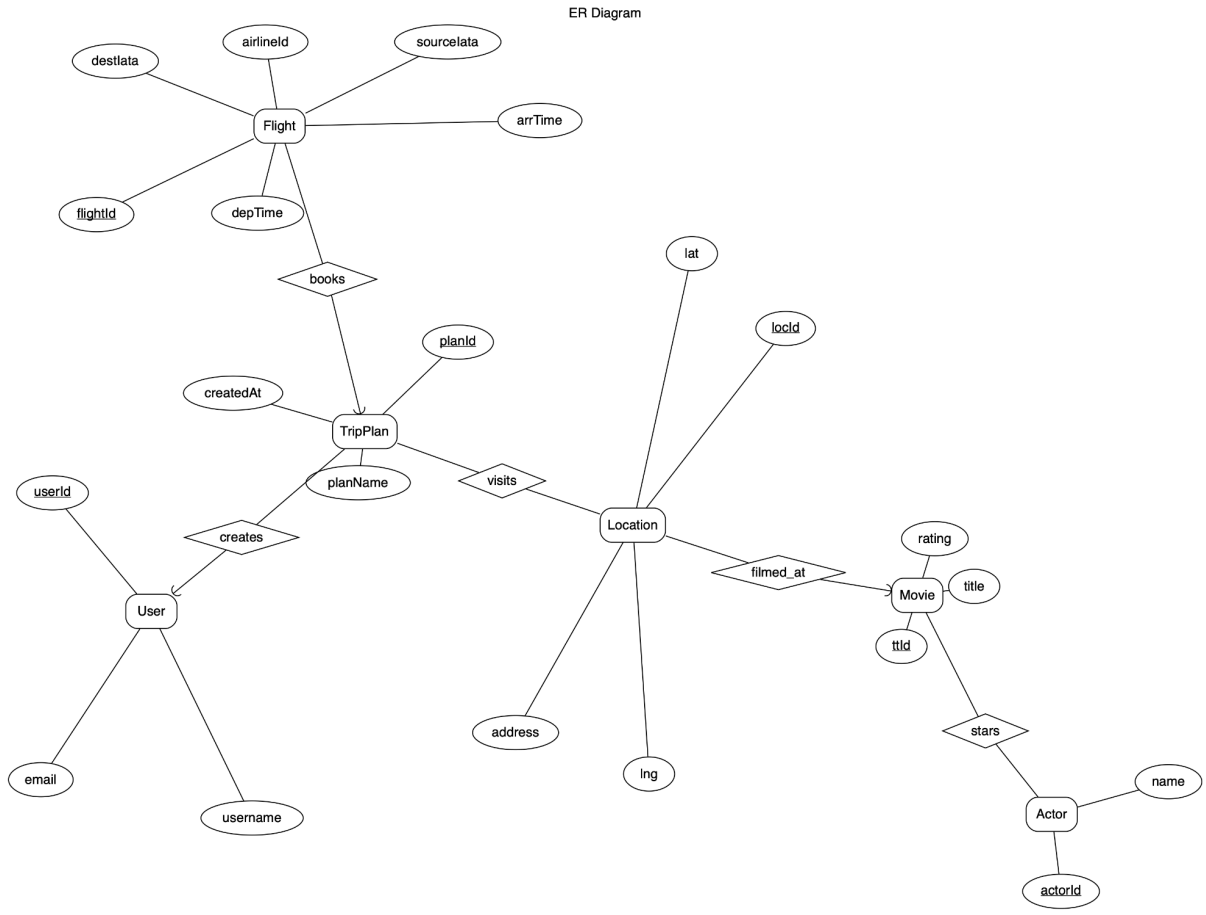
Stage 2: Database Design

1. ER Diagram

We have opted for an ER Diagram to represent our conceptual database design. The system centers around users creating travel plans based on movie filming locations, integrated with global flight routes.



---

2. Assumptions and Modeling Decisions

Entity Assumptions

- **User**: Represents account information. We limit this to one entity to comply with project constraints. It acts as the owner of travel plans.
- **Movie**: Based on the IMDb dataset. We modeled this as an entity rather than an attribute of Location because a movie has its own complex metadata (rating, title, cast) that would be redundant if repeated for every location.

- **Actor**: Modeled as a separate entity to support searching filming locations by specific cast members. This avoids storing actor names as comma-separated strings within the Movie table, which would violate 1NF.
- **Location**: Represents a movie-specific filming location record with address and coordinates. We assume a geocoding process can populate lat/lng for planning (and may remain NULL if geocoding fails) to support later TSP-style computations.
- **Flight**: Represents an itinerary leg between airports. Connectivity can be aligned with OpenFlights routes; depTime/arrTime are simulated fields for display/sorting at this stage and may be populated by APIs or estimation in later implementation.
- **TripPlan**: A central entity that aggregates a user's selected locations and the flights chosen to connect them.

Relationship and Cardinality

- **User to TripPlan (1:N)**: A single user can manage multiple "pilgrimage" itineraries, but each plan is uniquely tied to one account for privacy and data integrity.
- **Movie to Actor (M:N)**: A movie typically features multiple actors, and an actor can appear in many movies. This requires a junction table in the logical design.
- **Movie to Location (1:N)**: Each movie maps to multiple filming location records; each Location record belongs to exactly one movie since Location is defined as a movie-specific filming record rather than a globally unique place.
- **TripPlan to Location (M:N)**: A TripPlan can include many Location records, and the same Location record may appear in many users' plans (e.g., multiple users saving the same filming record from a movie).
- **TripPlan to Flight (1:N)**: A plan consists of a sequence of flight legs. We store the leg order via an itinerary/sequenceOrder relation (Plan_Itinerary in the logical design) to support display and reproducibility at this stage.

---

3. Normalization

Our schema adheres to **Third Normal Form (3NF)**:

1. **First Normal Form (1NF)**: All attributes contain atomic values. The stars list from the raw dataset has been decomposed into an Actors table and a junction table.
2. **Second Normal Form (2NF)**: All non-key attributes are fully functionally dependent on the primary key. In the Movies table, Title and Rating depend entirely on ttId.
3. **Third Normal Form (3NF)**: There are no transitive dependencies. For example, we do not store airport city names in the Flights table; instead, we use IataCode to reference the Airports (though only Flights is shown here for brevity, the logic holds).

---

4. Logical Design (Relational Schema)

- **Users**(UserId:INT [PK], Username:VARCHAR(50), Email:VARCHAR(100))

- **Movies**(ttId:VARCHAR(20) [PK], Title:VARCHAR(255), Rating:DECIMAL(3,1))
- **Actors**(ActorId:INT [PK], Name:VARCHAR(255))
- **Movie_Stars**(ttId:VARCHAR(20) [PK, FK to Movies.ttId], ActorId:INT [PK, FK to Actors.ActorId])
- **Locations**(LocId:INT [PK], ttId:VARCHAR(20) [FK to Movies.ttId], Address:VARCHAR(255), Lat:DECIMAL(9,6), Lng:DECIMAL(9,6))
- **Flights**(FlightId:INT [PK], AirlineId:VARCHAR(10), SourceIata:VARCHAR(5), DestIata:VARCHAR(5), DepTime:TIME, ArrTime:TIME)
- **TripPlans**(PlanId:INT [PK], UserId:INT [FK to Users.UserId], PlanName:VARCHAR(100), CreatedAt:DATETIME)
- **Plan_Itinerary**(PlanId:INT [PK, FK to TripPlans.PlanId], FlightId:INT [PK, FK to Flights.FlightId], SequenceOrder:INT)
- **Plan_Locations**(PlanId:INT [PK, FK to TripPlans.PlanId], LocId:INT [PK, FK to Locations.LocId])